Wallet & P2P Transfer

DESIGN, CONCURRENCY & OPERATIONS | 10 SEPTEMBER 2026

Model and scope. wallets stores a UUID, unique opaque user identity, non-negative BIGINT paise and timestamps. transfers stores UUIDs for both wallets, a globally unique bounded idempotency key, positive BIGINT amount, durable outcome/reason and timestamps. Checks reinforce distinct endpoints and outcome consistency; a deferred trigger rejects a pending outcome at commit. Input JSON numbers must be safe integers; all money responses are exact decimal strings. Bearer identities and first-create funding are deliberately simple exercise facilities. Only the winning wallet INSERT can seed a balance; replay cannot replenish it.

Concurrency choice. One READ COMMITTED PostgreSQL transaction reserves the unique transfer key, then locks both wallet rows using ORDER BY id FOR UPDATE. Authorization occurs under those locks. A conditional debit (WHERE balance_paise >= amount) and the matching credit commit with the final outcome. Insufficient funds and recipient BIGINT overflow produce durable declines without money movement. Sorted wallet locking gives A to B and B to A the same first lock. Crucially, both wallet foreign keys are deferred: their KEY SHARE checks happen at commit, when this transaction already holds both stronger locks. The earlier unique-index reservation therefore cannot introduce a wallet lock-order cycle. No in-memory mechanism controls correctness.

Idempotency and failure. Concurrent duplicate insertions wait for the winning transaction. After a committed winner, the loser rolls back and reads the stored result on its already-checked-out connection; even a one-connection pool progresses. Replay verifies the original source owner, then compares canonical UUIDs and amount. A valid changed request returns 409. A decline remains unchanged even after replenishment. Lost response after commit: retry the same key/body/identity to retrieve that result. An uncertain COMMIT connection failure is resolved the same way. Precommit failures roll back the key and both balances; unreliable cleanup discards the connection. GET transfer permits either participant; wallet reads only the owner.

Alternatives and trade-off. Read-then-write replacement loses concurrent updates. In-memory keys disappear on restart. Unsorted locks and immediate foreign-key checks can deadlock opposite transfers. Serializable isolation is viable but adds abort/retry handling without simplifying two explicit row locks; Redis locks, queues and distributed transactions add unnecessary components. This service chooses consistency over write availability during database outages and latency under contention. PostgreSQL remains the authority; direct administrator writes are trusted and out of the API contract.

Verification and operations. Real PostgreSQL tests cover authorization/validation, 50 concurrent creates, 30 identical transfers, 300 opposite-direction contended transfers, exact BIGINT balances, row counts, and 50 affordable competing debits against insufficient aggregate funds. Failure probes inject credit/commit exceptions, terminate backends, and drop an HTTP response after a real commit. The burst saves timestamped results with HTTP errors, transport failures/retries and client latency separately. Compose/CI check fresh installs, build, non-root production-only images, health and database counts. JSON logs and a bounded sanitized public feed expose committed outcomes; bounded Prometheus counters/histograms report rate, errors, domain outcomes and server p99. Logs/counters can be lost in the postcommit crash window; they are not an audit ledger. Executed results and source revisions are in docs/VERIFICATION.md.

AI disclosure. The user explicitly directed the stack, scope, invariants, operational deliverables, test thresholds, preservation of history, and autonomous routine decisions. This session's AI inspected and preserved an existing 14-commit implementation, chose the targeted fixes, tests, migration, runtime and evidence workflow, and used an independent AI reviewer. No new human architecture approval, unaided manual implementation or earlier test result is claimed. Historical Git attribution is unchanged.

Verified Rs. 0 limit. On September 10, 2026, the existing Render Docker API and managed PostgreSQL were Free, the workspace Hobby, with no card on file and $0 accrued. The free API sleeps after 15 idle minutes; its workspace has 750 hours/month. The 1 GB database expires October 10, 2026 and has no free backups. The account showed 5 GB/month bandwidth. With no payment method, quota exhaustion suspends service/builds. No paid resources were enabled. This is time-limited exercise hosting. Render terms.